

Centro Universitário Senac

Controle de acesso para o público PCD

**Gabriela Fernandes - BEC
Guilherme Akio Tamura – BEC
Joyce Nunes Alves – BEC
Matheus Biazio Spila - BEP**

São Paulo – SP
2025

Gabriela Fernandes
Guilherme Akio Tamura
Joyce Nunes Alves
Matheus Biazio Spila

Controle de acesso para o público PCD

Pré-projeto apresentado como requisito básico para a
apresentação do Projeto de PI.

Orientador (a): Marcello Marcellino

São Paulo – SP
2025

Sumário

1.	INTRODUÇÃO – TEMA E PROBLEMATIZAÇÃO	4
2.	JUSTIFICATIVA.....	6
3.	OBJETIVOS	7
4.	METODOLOGIA DA PESQUISA PARA O PROJETO	9
4.1	MODELAGEM DA CATRACA	9
4.2	SISTEMA ELETRÔNICO.....	12
4.3	PROGRAMAÇÃO E BANCO DE DADOS.....	20
4.4	APLICAÇÃO WEB.....	28
5.	CRONOGRAMA	37
6.	CONCLUSÃO	37
7.	REFERÊNCIAS	38

1. INTRODUÇÃO – TEMA E PROBLEMATIZAÇÃO

Este relatório apresenta o desenvolvimento de uma solução integrada de controle de acesso, que utiliza o reconhecimento facial como método de autenticação e inclui melhorias de acessibilidade para indivíduos com deficiência física. A solução proposta visa não apenas aumentar a segurança, mas também garantir que todos os usuários, independentemente de suas capacidades físicas, possam acessar os ambientes de forma autônoma e digna.

O projeto propõe uma catraca automatizada com reconhecimento facial, voltada a pessoas com deficiência (PCD), a fim de facilitar o acesso autônomo e seguro em ambientes controlados (ex: empresas, instituições de ensino). A catraca deve permitir a passagem automaticamente ao identificar um rosto previamente cadastrado.

O foco no público PCD é um diferencial importante, promovendo acessibilidade e autonomia. O uso de reconhecimento facial é moderno, evitando o uso de cartões, botões ou comandos físicos. Integração com banco de dados: o reconhecimento é feito a partir de rostos cadastrados, garantindo controle de acesso. Preocupação com segurança: há menção à necessidade de proteger os dados biométricos (LGPD).

Fluxo Geral do Sistema (Resumido)

Usuário se aproxima → câmera captura rosto.

Imagem é enviada para sistema de reconhecimento (Edge ou Azure).

Reconhecimento consulta banco (SELECT).

Se aprovado → retorna REPULT para ESP.

ESP ativa motor da catraca.

Tudo isso pode ser monitorado por uma interface em Replit, que também envia INSERT/DELETE.

A crescente demanda por soluções tecnológicas inclusivas evidencia a importância de sistemas que combinem segurança, praticidade e acessibilidade. Neste contexto, o presente pré-projeto propõe o desenvolvimento de um sistema de controle de acesso inteligente por meio de catraca automatizada, com foco específico no atendimento a pessoas com deficiência (PCD). A proposta se baseia no uso de tecnologias emergentes, como reconhecimento facial, integração com banco de dados em nuvem e dispositivos embarcados (ESP32), garantindo autonomia ao usuário e robustez ao sistema.

O projeto tem como objetivo criar uma catraca com acionamento automático a partir da autenticação facial, eliminando a necessidade de contato físico ou uso de

cartões, o que representa uma inovação significativa em acessibilidade. A interface será apoiada por uma aplicação web para administração e visualização de acessos em tempo real, enquanto todo o fluxo eletrônico será estruturado para operar de forma segura, escalável e com baixo consumo energético. A proposta integra princípios de Internet das Coisas (IoT), computação de borda (Edge Computing) e conceitos de engenharia aplicada, consolidando uma solução multidisciplinar voltada ao bem-estar social.

2. JUSTIFICATIVA

A escolha do reconhecimento facial como método de autenticação se justifica pela sua eficiência e segurança. Diferente de métodos tradicionais, como senhas e cartões de acesso, o reconhecimento facial é difícil de falsificar e oferece uma experiência de usuário mais fluida. Além disso, a integração de funcionalidades de acessibilidade é essencial para garantir a inclusão de pessoas com deficiência física, promovendo a igualdade de acesso e a autonomia desses indivíduos.

O desenvolvimento da catraca utilizando a placa OPS010 e a integração com um sistema de reconhecimento facial são passos fundamentais para a criação de um sistema confiável e eficiente. A implementação de um módulo de cadastro de usuários com imagens faciais e a criação de uma interface web para visualização dos acessos são componentes críticos para a gestão eficiente do sistema. A integração segura entre o controle de acesso e a base de dados da empresa garante a sincronização das informações e a proteção dos dados dos usuários.

Por fim, a realização de testes de segurança e usabilidade é crucial para validar a eficácia da solução proposta. Esses testes garantirão que o sistema não apenas atenda aos requisitos de segurança, mas também seja acessível e fácil de usar para todos os usuários.

3. OBJETIVOS

3.1 Geral

Desenvolver uma solução integrada de controle de acesso, utilizando reconhecimento facial como método de autenticação e acessibilidade aprimorada para indivíduos com deficiência física, com visualização de dados em uma aplicação web.

3.2 Específicos

Desenvolver a catraca utilizando a placa OPS010.

1. Definir as especificações técnicas para a catraca, considerando o controle de acesso e os requisitos de integração com o sistema de reconhecimento facial.

2. Integrar a catraca com um sistema de autenticação via reconhecimento facial.

- Pesquisar e implementar um algoritmo de reconhecimento facial (pode ser utilizando bibliotecas como OpenCV, por exemplo).
- Garantir que o sistema de reconhecimento facial possa ser integrado com o sistema de cadastro de usuários de uma empresa.

3. Desenvolver a funcionalidade de cadastro de usuários com imagens faciais no sistema.

- Criar um módulo que permita o cadastro de novos usuários, associando suas imagens faciais a uma base de dados.
- Implementar a atualização ou exclusão de cadastros de forma fácil e acessível.

4. Garantir a acessibilidade para pessoas com deficiência física.

- Adaptar a interface de interação com o sistema de controle de acesso, considerando as necessidades de acessibilidade (como altura da catraca, dispositivos de apoio, etc.).
- Implementar recursos adicionais, como áudio ou feedback visual para auxiliar no processo de autenticação.

5. Desenvolver a interface web para visualização dos acessos.

- Criar uma aplicação web onde as informações de acessos (entrada/saída, horários, identificação de quem passou) possam ser visualizadas e gerenciadas em tempo real.
- Implementar filtros e relatórios para facilitar o acompanhamento e

análise dos acessos.

6. Implementar uma integração entre sistemas para a comunicação entre o controle de acesso e o sistema de cadastro de usuários da empresa.

- Definir uma forma segura e eficiente de integrar o sistema de controle de acesso com a base de dados da empresa para garantir a sincronização das informações de cadastro.

7. Testar e validar a solução com foco em segurança e usabilidade.

- Realizar testes de segurança para garantir que a autenticação facial seja confiável e que as informações estejam protegidas.

- Fazer testes com usuários reais para verificar a eficácia da autenticação facial e a acessibilidade da solução.

4. METODOLOGIA DA PESQUISA PARA O PROJETO

O Projeto busca desenvolver um controle de acesso para o público PCD por meio de pesquisa de material técnico sobre os softwares, banco de dados e placas e trabalho operacional para desenvolvimento do equipamento por meio de testes e implementação.

Esse projeto consiste em uma pesquisa experimental. Ocorre quando manipula-se diretamente as variáveis relacionadas com o objeto de estudo. A manipulação de variáveis proporciona o estudo da relação entre as causas e os efeitos de determinado fenômeno. (CERVO; BERVIAN; DA SILVA, 2013, p.61). Para Gil (2010, p.73), “de modo geral, o experimento representa o melhor exemplo de pesquisa científica”.

Utilizaremos manuais de uso prático para utilização da LM2596, Arduino IDE, Placa OPS010 e esp32 com objetivo desenvolver uma solução integrada de controle de acesso, utilizando reconhecimento facial como método de autenticação e acessibilidade aprimorada para indivíduos com deficiência física. Também será necessário criar uma aplicação Web que exiba os dados de entrada e saída obtidos da implementação.

4.1 MODELAGEM DA CATRACA

Foi desenvolvido uma portinhola 50x50 que contará dois chanfros na parte superior, de modo que possa ser possível acoplar dois módulos de esp32_cam para reconhecimento facial. Esses componentes serão os responsáveis por enviar a informação ao controlador OPX001 e abrir a catraca.

O giro será feito 90° a esquerda para fechar e 90° a direita para fechar. Abrir peça(s) e outro PC para pegar print das imagens e colocar no documento. O modelo terá um suporte no chanfro, para acoplar o espcam na catraca.

Materiais necessários: SolidWorks, Placa MDF (dimensões mínimas: 70x70) e Parafuso Alien.

Estrutura Física da Catraca

No canto direito do quadro, há o desenho esquemático da catraca física com a instalação do ESP.

Fluxo:

ESP está dentro da estrutura, comandando o atuador.

Recebe comando de abertura após reconhecimento facial bem-sucedido.

A peça contém quatro círculos com fendas internas (tipo "raia").

Está dentro de um contorno retangular, provavelmente um recorte de teste.

Isso é comumente usado para testar encaixe de eixos, motores ou trancas.

As fendas centrais podem ser entradas para "chavetas", pinos-guia ou até eixos planos.

Útil para verificar tolerância do corte a laser e ajuste mecânico.

Use esta peça em MDF ou acrílico para testar se o motor ou pino trava corretamente dentro da catraca antes da montagem final.

Peça retangular com 3 furos circulares (um central grande e dois pequenos simétricos).

Bordas arredondadas sugerem que é parte visível ou de apoio fixo.

Essa peça serve como base ou flange de fixação para o motor de redução.

O furo central acomoda o eixo do motor, e os furos menores são para parafusos de fixação.

Ideal para montagem frontal em uma das paredes internas da caixa da catraca.

Recomenda-se fixá-la com parafusos M3 ou M4.

Verifique se o centro do furo grande está alinhado com o ponto de contato com o braço da catraca.

Grande peça com múltiplos entalhes simétricos e um furo quadrado no centro.

Tem recortes retangulares verticais laterais e encaixes tipo "macho-fêmea".

Esta é claramente uma das laterais da caixa da catraca.

O recorte central quadrado pode ser para a passagem do eixo do motor ou de fios.

Os recortes laterais são encaixes para união com as demais faces da caixa.

Utilize este DXF para corte em MDF 3mm ou 6mm.

O projeto parece ser de encaixe por pressão ou com cola, sem uso de parafusos visíveis.

Recomendações Gerais para o Corte a Laser

Item	Recomendação Técnica
Material sugerido	MDF 3mm ou 6mm, ou acrílico
Espessura compatível	Compatível com os encaixes vistos no arquivo DXF
Tipo de montagem	Por pressão (encaixe), talvez com cola de madeira
Tolerância de corte	Considere 0.2 mm de folga lateral para encaixes

Item	Recomendação Técnica
Teste de encaixe	Use o TESTEENCAIXE.DXF primeiro

Montagem 3D Final

A imagem mostra a montagem completa da caixa da catraca.

Os encaixes tipo “macho-fêmea” estão bem distribuídos para garantir trava estrutural sem parafusos.

A geometria é modular e ideal para corte a laser em MDF.

A aba à esquerda lista os componentes como PARTE...CAIXACATRACA e a base.

Esboço da Parede com Furo Central

Um retângulo 40x56mm com centro marcado.

Provavelmente a parede lateral frontal ou traseira da caixa.

O furo central (outra operação) pode ser para o eixo do motor ou passagem de fiação.

Base com 4 Furos

Esboço com furações: 4 furos com diâmetro de 5 mm distribuídos simetricamente.

Indicação clara de que será a base para fixação da caixa no chão ou sobre um suporte.

Os furos estão equidistantes (20 mm entre centros), facilitando alinhamento na montagem.

Peça extrudada com 4 furos

Resultado da extrusão da base (esboço da imagem anterior).

Peça plana com 4 furos, possivelmente com extrusão de 3 a 6 mm.

Representa a base estrutural onde serão montadas as paredes verticais da catraca.

Posicionamento de encaixes

Processo de inserção de restrições (mates) entre partes.

O tipo de posicionamento usado é "Coincidente", alinhando superfícies planas ou arestas.

Mostra que os encaixes laterais estão sendo ajustados com precisão (1 mm visível).

Caixa semi-montada com vista explodida

Vista lateral com a estrutura sendo montada parcialmente.

A peça superior está deslocada, evidenciando a estrutura do interior da caixa.
Ideal para documentar montagem em etapas ou gerar manual de instruções.

Ajuste dimensional da parede

Esboço mostra as dimensões definidas para uma das laterais da caixa: 56 mm de altura.

Você está ajustando o comprimento para garantir encaixe perfeito com a base e topo.

Aspecto	Observação
Montagem no SolidWorks	Bem organizada, com uso correto de restrições e nomes de peças
Estrutura da caixa	Modular, com encaixes que dispensam uso de parafusos (ideal para MDF)
Fixação da base	Prevista com furos simétricos para garantir estabilidade
Planejamento de corte	Geometrias com cantos retos e medidas simples indicam boa otimização de chapa
Boas práticas de modelagem	Peças separadas por função e nomeadas corretamente no ambiente de montagem

4.2 SISTEMA ELETRÔNICO

O projeto atualmente está contando com duas placas ESP32, sendo que uma delas possui uma câmera para acoplamento, um CLP OPX001, que possui input's/output's de 24 e 10v, com as portas de 10v sendo usadas para conexão com o ESP, dois sensores ultrassônicos, que serão utilizados para câmera identificar que precisa fazer um reconhecimento e para catraca fechar após a passagem de alguém (o alcance do sensor é de 50cm). Também será usado um motor de redução para controlar a velocidade de rotação da portinhola.

Materiais necessários: Fios, Ferro de solda, Estanho, Resistores, Jumper's, Protoboard, ESPcam32, ESP32, Um sensor Ultrassônico, dois led's, Servo Motor, Fonte chaveada (24v) e Fonte pequena (3.3V).

A ideia geral é que na catraca fique acoplados os dois Esp's, que ficaram responsáveis por captar a informação que servirá para acionar a catraca (seja para entrada ou saída). É possível fazer toda a comunicação via wi-fi através do ESP

entretanto, a comunicação entre o esp e o CLP deverá ser feita através das ligações com as portas A0/ 0-10V.

ESPCAM — Conecta a internet > Identifica um registro > manda a informação de que identificou uma pessoa para o EDGE IMPULSE > Na validação do EDGE Impulse será incluído um registro no Banco de Dados.

Banco de Dados - Do EDGE Impulse será enviada uma comunicação para o PHP que se comunicará com o Banco de Dados.

ESP32 x OPS - Com o código que será desenvolvido para o ESP, ele vai ler o banco de dados a fim de identificar algum sinal para enviar para o OPX e o braço da catraca ser ativado.

O circuito que fará a comunicação ESP-OPX terá como objetivo realizar o acionamento de uma entrada recebendo uma informação através de um sinal analógico.

Para o sistema eletrônico serão utilizados Microcontroladores/atuadores (para abertura da catraca), Câmera acoplada ao sistema, Motor ou atuador para girar a catraca e Fonte de energia.

Processamento com ESP-CAM / ESP32

A imagem facial é capturada por uma ESP32-CAM, que envia os dados via:

Wi-Fi ou Bluetooth

Para um servidor na nuvem (CL) ou local

Depois, o resultado do reconhecimento (autorizado/não autorizado) volta via MQTT, que é um protocolo leve de comunicação IoT.

Está sugerido que o ESP-CAM apenas coleta e envia a imagem.

O ESP32 separado recebe o comando final e aciona a catraca, usando GPIOs.

Controle Eletrônico da Catraca

No centro e à direita:

ESP32 ativa um transistor que controla um relé ou driver para liberar a catraca.

Tensão de +20V ou 24V é fornecida à catraca.

A corrente está dimensionada para 1A, o que indica um atuador/motor de porte leve.

O acionamento é feito por um sinal digital (I/O) vindo do ESP32.

Sensor de Passagem

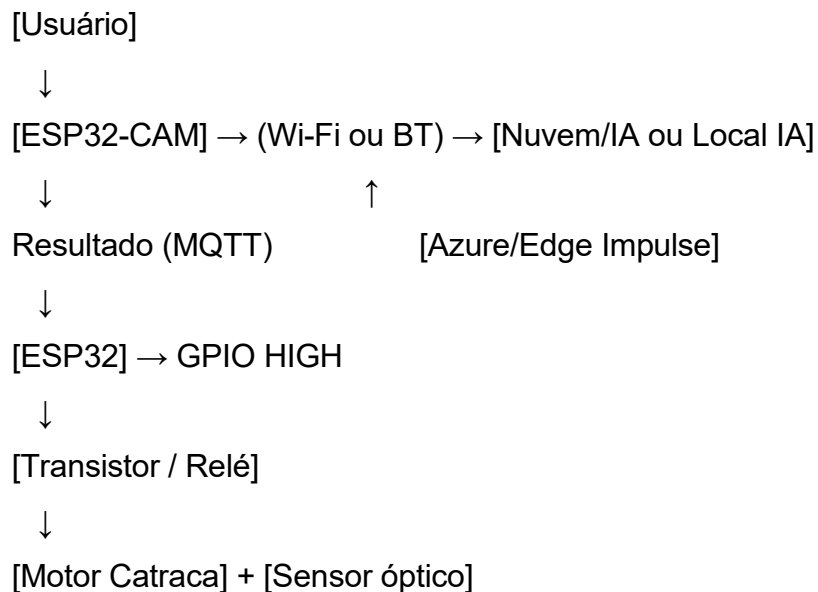
Foi desenhado um esquema com sensor óptico (fototransistor + LED infravermelho).

Esse sensor detecta se alguém passou pela catraca após o acionamento.

O sinal é processado novamente pelo ESP32 como uma confirmação de

passagem, útil para logs ou falha de abertura.

Componente	Função
ESP32-CAM	Captura facial e envia para nuvem (ou IA local)
ESP32 (separado)	Recebe autorização e aciona a catraca (comanda relé/transistor)
Transistor (ou relé)	Intermediário entre ESP e motor da catraca
Fonte 24V/1A	Alimenta o motor da catraca
Sensor óptico (IR)	Verifica se o usuário passou após a liberação
MQTT	Protocolo de comunicação leve entre dispositivos e nuvem



Identifique os Componentes e Suas Tensões

Componente	Tensão (V)	Corrente típica (A)	Observações
ESP32 / ESP32-CAM	5V (ou 3.3V via regulador)	0,25 – 0,5 A	Alimentação via conversor (step- down)
Sensor óptico	5V	~0,02 A	Alimentado

Componente	Tensão (V)	Corrente típica (A)	Observações
(IR)			diretamente do ESP32

Fonte Chaveada 24V / 1A ou 2A

Ideal se a catraca utiliza 24V para o solenoide/motor.

Pode usar um buck converter (step-down) para reduzir de 24V para 5V e alimentar o ESP32.

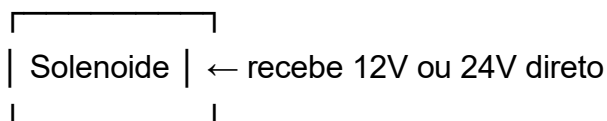
Exemplo: Fonte 24V 2A + conversor LM2596 (ajustável) → 5V.

Esquema de Ligação

[Tomada AC 110/220V]



[Fonte Chaveada 12V ou 24V]



[Conversor Step-down LM2596]



[ESP32 via pino VIN (5V)]

Use uma fonte com sobra de corrente (ex: se motor consome 1A, use fonte de 2A).

Evite fontes USB de celular para alimentar motor/relé – não aguentam picos de corrente.

Prefira fonte chaveada com saída estabilizada, como essas de projetos Arduino.

Se sua catraca usa um solenoide 24V/1A, a solução ideal seria:

Fonte chaveada 24V 2A

Conversor LM2596 ajustado para 5V

ESP32 alimentado pelo conversor

Motor acionado via relé controlado pelo ESP32

Tipo de motor	Exemplo comum em catraca?	Driver recomendado
Motor DC	Às vezes	L298N, L9110, TB6612FNG

Driver L298N (motor DC)

Permite controle de direção e velocidade (PWM).

Útil se sua catraca tiver motor bidirecional.

Funciona bem com motores de até 2A.

Pode alimentar o motor e o ESP32 (via regulador 5V).

E quanto ao Shield para “Placa OPS”?

Se por “placa OPS” você está se referindo à placa principal do projeto onde estão os conectores e controladores (como o ESP32 e sensores), então o driver escolhido (ex: relé ou IRF520) deve ser integrado à essa placa, com:

Pino de controle vindo do ESP32 (GPIO).

Fonte separada para o motor (12V ou 24V).

Proteção com diodo flyback, especialmente para solenoides e motores DC.

Resumo do Motor

Tipo: Motor DC pequeno (não é de passo nem solenoide).

Tensão: 3V a 6V

Corrente: até 1A (presumido, depende do motor — se tiver o datasheet, ótimo)

Driver	Tensão Motor	Corrente	Direção	PWM (velocidade)	Compatível ESP32
L298N	≥ 6V (real)	até 2A	✓	✓	⚠ (queda de tensão alta)

Fonte para motor 3V a 6V

Fonte externa de 5V ou 6V com corrente $\geq 1A$.

Ou use pilhas recarregáveis (NiMH) ou pack de baterias Li-ion 2S (7.4V com step-down).

Motor DC (3V–6V) aciona a catraca.

Dois REED switches como fins de curso (limite esquerdo e direito).

Motor deve desligar automaticamente quando um dos REEDs for acionado.

Controle feito por ESP32 + ponte H.

Como usar os fins de curso (REED switches)?

Opção A – Leitura via ESP32 (Recomendada):

Os dois REED switches vão para entradas digitais do ESP32.

O ESP32 monitora constantemente os estados desses pinos.

Quando um dos REEDs é ativado (ímã se aproxima), o ESP:

Envia $IN1 = IN2 = LOW \rightarrow$ desliga o motor.

Ou aplica "freio eletrônico" (dependendo da lógica).

Esquema lógico básico:

REED A (GPIO 12) $\leftarrow$ Detecta fim no sentido horário

REED B (GPIO 13) $\leftarrow$ Detecta fim no sentido anti-horário

ESP32:

- Enquanto nenhum REED ativado $\rightarrow$ gira motor

- Se REED A ativado $\rightarrow$ para motor (sentido A)

- Se REED B ativado $\rightarrow$ para motor (sentido B)

ESP32 (na placa OPX001)

Função: Controlador central do sistema. Conecta sensores, REED switches, motor, faz a lógica de decisão e comunicação com o Azure.

Na prática:

Recebe comando da ESP-CAM (via Wi-Fi ou serial).

Aciona motor (via ponte H) e lê os REED switches.

Conecta à internet (para Azure ou broker MQTT).

Importante: opera com 3.3V! Não pode ser alimentado direto com 24V.

ESP32-CAM

Função: Captura a imagem do rosto e envia para o processamento facial.

No projeto:

Roda interface de câmera (como mostra a imagem com IP 192.168.x.x).

Pode enviar resultado para o ESP32 via Wi-Fi ou serial.

Atua como sensor biométrico autônomo.

Sensores ultrassônicos (HC-SR04)

Função: Detectar aproximação do usuário da catraca ou passagem (redundância com REEDs).

Um sensor pode acionar a câmera quando alguém se aproxima.

Outro pode confirmar se o usuário passou após o motor abrir.

Fonte Chaveada LUCKFOYU 24V 2A

Função: Alimenta o motor da catraca e outros componentes.

Especificações:

Entrada: 110–220V AC

Saída: 24V DC / 2A (até 50W)

Não pode alimentar diretamente o ESP32 → você usará um step-down para isso.

Pode alimentar o motor de redução e a ponte H diretamente.

Step-down LM2596

Função: Reduzir de 24V para 5V (para alimentar o ESP32).

Conecte a entrada (VIN) nos 24V da fonte chaveada.

Regule a saída para 5.0V com multímetro antes de ligar ao ESP32.

Conecte OUT+ ao pino VIN do ESP32 e OUT- ao GND.

Faça a regulação antes de conectar qualquer circuito sensível.

Driver Ponte H L298N

Função: Controlar o motor de redução 3–6V.

Tem queda de tensão interna (~2V), então se alimentar com 6V, pode sair ~4V para o motor.

Não alimente a ponte com 24V direto!

Use o LM2596 para baixar de 24V para 6V ou 5V e conectar no terminal VCC da ponte H.

IN1, IN2, EN → ligados a GPIOs do ESP32.

Motor de Redução 3V–6V

Função: Atua como mecanismo de liberação da catraca (gira ou empurra o braço).

Conectado aos pinos OUT1 e OUT2 do driver L298N.

Recebe comando do ESP32 para girar em um dos sentidos até o fim de curso (REED).

REED Switches

Função: Fins de curso. Parar o motor ao atingir posição aberta ou fechada.

Ligados a entradas digitais do ESP32 com INPUT_PULLUP.

Detectam presença de ímã fixado no eixo da catraca.

Usados para interromper o giro do motor automaticamente.

Protoboard 830 pontos

Função: Montagem temporária dos circuitos sem solda.

Ligar os REEDs, sensores ultrassônicos, LM2596 e ponte H.

Testar lógica antes de soldar definitivamente.

Jumpers e fios desencapados

Função: Conectar os componentes.

Fios rígidos (tipo jumper) para protoboard.

Fios flexíveis desencapados ou soldados para a placa OPX001, sensores e alimentação final.

Evite ligar motores diretamente na protoboard — solda ou bornes são mais seguros.

Diagrama Geral Simplificado (Conceitual)

[Tomada AC 110/220V]



[Fonte 24V 2A]

└─→ [Motor via L298N]

└─→ [LM2596 step-down] → 5V → ESP32

ESP32:

└─→ Comanda L298N (IN1, IN2, ENA)

└─→ Lê REED Switch A e B

└─→ Lê sensores ultrassônicos

└─→ Comunicação com ESP32-CAM ou Broker MQTT

ESP32-CAM:

└─→ Captura imagem

└─→ Interface web ou reconhecimento facial local

CÓDIGO ESP32

```
int IN1 = 12; // Controle de direção 1
```

```
int IN2 = 13; // Controle de direção 2
```

```
void setup() {
```

```
  pinMode(IN1, OUTPUT);
```

```
  pinMode(IN2, OUTPUT);
```

```
  // Inicialmente parado
```

```
  digitalWrite(IN1, LOW);
```

```
  digitalWrite(IN2, LOW);
```

```

}

void loop() {
    bool rostoReconhecido = true; // simule por enquanto

    if (rostoReconhecido) {
        digitalWrite(IN1, HIGH); // Gira em um sentido
        digitalWrite(IN2, LOW);

        delay(3000); // gira por 3 segundos (ajuste conforme o necessário)

        // Parar motor
        digitalWrite(IN1, LOW);
        digitalWrite(IN2, LOW);
    }

    delay(10000); // evita reativar logo em seguida
}

```

4.3 PROGRAMAÇÃO E BANCO DE DADOS

Banco de Dados — o escopo das tabelas para armazenamento das informações de entrada e saída. Cadastro e alimentação dos dados. Realização de armazenamento de imagens para treinar a Inteligência Artificial.

Uso do EDGE Impulse, uma API para treinar a IA e implementa-la em uma lógica com o Python. Foram estudada as linguagens e plataformas como o Python com OpenCV (biblioteca comum para reconhecimento facial) e banco de dados para armazenar rostos autorizados, gerando Relatório de acessos, inclusos logs de acessos.

O banco de dados é crucial para armazenar os rostos autorizados e informações do usuário, como: ID do usuário, Nome, Imagem facial codificada (vetores), Data/hora do último acesso, Permissões (ex: restrição de horários), etc.

O Azure pode ser utilizado de várias formas, dependendo da necessidade e orçamento: Banco de dados em nuvem (Azure SQL Database): escalável e seguro, Reconhecimento facial com Azure Face API: caso queiram usar uma API já treinada e com suporte, Hospedagem de interface/admin: criar painel web em Azure App Services. e Segurança e autenticação: controle de usuários via Azure Active Directory.

Edge Impulse é uma plataforma de aprendizado de máquina para dispositivos embarcados (como Arduino, ESP32, Raspberry Pi). Pode ser usada para:

Treinar um modelo de reconhecimento facial leve, rodando localmente (edge computing).

Otimizar modelos para baixa latência e consumo de energia reduzido.

Coletar dados de imagens faciais, rotular e treinar com base nesses dados.

Relação com o projeto:

Ideal se o projeto quiser fugir de APIs na nuvem (por custo ou privacidade) e rodar o reconhecimento direto no dispositivo embarcado, mesmo offline. Isso melhora a resposta em tempo real e protege os dados, pois tudo fica local.

Reconhecimento Facial: É o coração da catraca. A câmera capta o rosto do usuário, o software processa e:

Compara com os rostos cadastrados no banco (via vetores ou hashes).

Decide se abre ou não a catraca.

Implementação possível:

Python + OpenCV para detecção facial.

Face recognition (dlib) para codificação e comparação.

Comunicação com o atuador da catraca via GPIO ou relé (ex: usando Raspberry Pi ou ESP32).

O sistema deve garantir boa acurácia, mesmo em variações de iluminação, expressões e PCDs com características faciais diversas. Pode ser necessário treinar o modelo com exemplos variados de cada pessoa (de frente, com óculos, etc).

graph TD

Camera --> ReconhecimentoFacial

ReconhecimentoFacial --> BancoDeDados

BancoDeDados -->|verifica| Resultado[Usuário autorizado?]

Resultado -->|sim| Atuador[Abre catraca]

Resultado -->|não| Negado[Não abre]

ReconhecimentoFacial --> Azure[(Azure Face API)]:::cloud

BancoDeDados --> AzureSQL[(Azure SQL Database)]:::cloud

classDef cloud fill:#cde,stroke:#369;

Esses 4 pontos são altamente complementares e podem formar um sistema moderno, funcional e escalável:

BD → armazena identidades

Azure → conecta, protege e escala

Edge Impulse → otimiza o processamento local

Reconhecimento facial → executa a autenticação

Dispositivo (ESP)

Representa o microcontrolador no hardware da catraca (ESP32, por exemplo).

Ele é responsável por capturar os dados (imagem/facial) ou receber a confirmação de reconhecimento facial via rede.

Envia INSERT de dados para o banco (ex: quando novo usuário é cadastrado).

Recebe comandos de abertura da catraca (REPULT → "resultado").

Reconhecimento Facial (Edge / Microsoft)

Refere-se ao uso de uma plataforma de IA como o Edge Impulse ou Azure Face API para realizar o reconhecimento facial.

Realiza o processamento facial com a imagem capturada.

Consulta o banco de dados (via SELECT) para verificar se o rosto é autorizado.

Retorna o resultado do reconhecimento (REPULT) ao ESP.

Banco de Dados (Azure SQL ou local)

O banco central onde estão os usuários cadastrados, imagens codificadas, logs de acesso, etc.

Operações:

INSERT: cadastro de novo rosto/usuário.

SELECT: consulta se um rosto é válido.

DELETE: remoção de um usuário.

Conectado ao reconhecimento facial e ao ESP.

Entrada com Reconhecimento Facial

No canto superior esquerdo:

Um rosto sorridente indica o usuário → fornece "informação facial".

Essa informação facial é processada por um sistema de IA, que pode estar na nuvem ou local.

Integração com IoT e IA

No centro inferior:

A pergunta "IA p/ IoT?" sugere uma reflexão do professor sobre rodar a IA na borda (edge computing).

Pode ser com Edge Impulse no ESP32 ou Raspberry Pi, ou com reconhecimento facial diretamente na ESP-CAM (embora limitada).

Criar um Banco de Dados SQL

No portal, clique em "Criar um recurso".

Procure por SQL Database.

Clique em "Criar".

Preencha as informações:

Nome do banco: por exemplo, catracaDB

Servidor SQL: clique em "Criar novo" e configure um:

Nome do servidor (ex: servidorcatraca)

Login admin (ex: adminuser)

Senha segura

Região (ex: Brazil South)

Camada de preço: selecione Gratuito ou Básico (5 DTUs) para testes

Clique em "Revisar + Criar", depois em "Criar".

Permitir Conexões Externas (Firewall)

Depois que o banco for criado:

Vá até o servidor SQL criado.

Clique em "Firewalls e redes virtuais".

Adicione seu IP atual (clique em "Adicionar IP do cliente atual").

Salve as alterações.

Dados de Conexão

Anote estas informações:

Servidor: seu-servidor.database.windows.net

Banco: catracaDB

Usuário: adminuser@seu-servidor

Senha: (a que você escolheu)

Conectar via Azure Data Studio, VS Code ou Python

```
import pyodbc
```

```
server = 'seu-servidor.database.windows.net'
```

```
database = 'catracaDB'
```

```
username = 'adminuser'
```

```
password = 'suaSenhaForte'
```

```
driver = '{ODBC Driver 17 for SQL Server}' # instale se ainda não tiver
```

```
conn = pyodbc.connect(
```

```
f'DRIVER={driver};SERVER={server};PORT=1433;DATABASE={database};UID={user  
name};PWD={password}'
```

```
)
```

```
cursor = conn.cursor()
```

Exemplo: criar tabela

```

cursor.execute("""
CREATE TABLE usuarios (
    id INT PRIMARY KEY IDENTITY,
    nome NVARCHAR(100),
    imagem_vetor NVARCHAR(MAX),
    data_cadastro DATETIME DEFAULT GETDATE()
)
""")
conn.commit()

```

Integrar com Replit, ESP32 ou Backend

Replit → use o mesmo código Python acima, com pyodbc ou sqlalchemy.

ESP32 → a melhor prática é enviar os dados para um API REST que você hospeda, e ela se comunica com o banco.

Power BI / Visual Studio → conectam direto com o banco se preferir análise visual.

Código básico de lógica (Arduino/ESP32)

```
#define REED_A 12
```

```
#define REED_B 13
```

```
#define IN1 25
```

```
#define IN2 26
```

```

void setup() {
    pinMode(REED_A, INPUT_PULLUP);
    pinMode(REED_B, INPUT_PULLUP);
    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
}

```

```

void loop() {
    if (digitalRead(REED_A) == LOW) {
        // Parar motor ao atingir limite A
        digitalWrite(IN1, LOW);
        digitalWrite(IN2, LOW);
    } else if (digitalRead(REED_B) == LOW) {
        // Parar motor ao atingir limite B
        digitalWrite(IN1, LOW);
    }
}

```



```

    digitalWrite(IN2, LOW);
} else {
    // Motor girando sentido horário (exemplo)
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
}
}

```

Painel do Azure SQL Database

Essa imagem mostra a interface do portal Microsoft Azure, onde você já tem um banco SQL configurado com as seguintes características:

Item	Detalhes
Nome do banco de dados	Catracabd
Nome do servidor	servidorcatraca.database.windows.net
Grupo de recursos	Projeto-PCD
Status	 Online
Localização	BR Brazil South
Camada de preço	Uso Geral – Sem servidor (modelo serverless , Gen5, 1 vCore)
Plano	Azure for Students
Firewall configurável	Você pode abrir acesso clicando em “Definir o firewall do servidor”

O banco está ativo e pronto para uso com acesso externo (se IP estiver liberado).

O modelo serverless economiza custo, pois só consome recursos quando o banco é acessado (ideal para testes e projetos como este).

Você pode acessar com ferramentas como Azure Data Studio, Visual Studio Code, Beekeeper Studio, DBeaver ou diretamente com scripts Python, Node.js, etc.

Beekeeper Studio com tabelas do projeto

Essa imagem mostra o Beekeeper Studio aberto em um banco chamado

engenharia_26 com 4 tabelas e um trigger. Isso parece ser o modelo lógico da aplicação ou o banco local antes da migração para Azure.

Tabelas exibidas:

TB_ACESSO – Provavelmente guarda logs de entrada e saída dos usuários.

TB_ACOMPANHANTE – Provável vínculo de acompanhantes com usuários PCD.

TB_CADUNICO – Cadastro principal dos usuários (nome, CPF, rosto codificado, etc.).

TB_LOGTEMP – Logs temporários, talvez para testes ou auditoria de falha.

Trigger ativa na tabela TB_ACESSO

Nome da trigger: tr_tb_acesso_pri... (o nome está cortado)

Tipo: BEFORE INSERT

Linguagem: SQL padrão

Função: Disparar uma ação antes de inserir um novo acesso (por exemplo: validar duplicidade, preencher data_hora, inserir log, etc.)

Local	Função no Projeto
Azure SQL (catracabd)	É o banco em produção/nuvem , onde o sistema da catraca vai se conectar.
Banco local (engenharia_26)	É o banco que você usa para testes e modelagem local , com as tabelas reais.
Beekeeper Studio	Ferramenta para gerenciar as tabelas localmente ou na nuvem

Migrar as tabelas do banco local (engenharia_26) para o banco catracabd no Azure:

Use um script CREATE TABLE + INSERT via Beekeeper, Azure Data Studio ou Python.

Garanta que os tipos de dados e constraints estão iguais.

Reproduzir as Triggers no Azure:

Azure SQL suporta triggers normalmente.

Copie o conteúdo da trigger tr_tb_acesso_... e execute via console no Azure.

Conectar o sistema (ESP32 ou backend) ao banco catracabd:

Configure o IP no firewall.

Use a string de conexão disponível na opção “Mostrar cadeias de conexão do

banco”.

Código Banco de Dados

-- Criação do banco

```
CREATE DATABASE IF NOT EXISTS engenharia_26;  
USE engenharia_26;
```

-- Tabela: TB_CADUNICO

```
CREATE TABLE TB_CADUNICO (  
    ID_VISITANTE INT PRIMARY KEY,  
    NOME VARCHAR(100),  
    DATA_NASCIMENTO DATE,  
    NUM_CADASTRO_INCLUSAO VARCHAR(50),  
    GENERO ENUM('masculino', 'feminino', 'outro'),  
    POSSUI_ACOMPANHANTE TINYINT(1),  
    CAMINHO_FOTO VARCHAR(255),  
    DATA_CADASTRO TIMESTAMP  
);
```

-- Tabela: TB_ACOMPANHANTE

```
CREATE TABLE TB_ACOMPANHANTE (  
    ID_ACOMPANHANTE INT PRIMARY KEY,  
    ID_VISITANTE INT,  
    NOME_ACOMPANHANTE VARCHAR(100),  
    CONTATO VARCHAR(20),  
    OBSERVACAO TEXT,  
    FOREIGN KEY (ID_VISITANTE) REFERENCES  
TB_CADUNICO(ID_VISITANTE)  
);
```

-- Tabela: TB_ACESSO

```
CREATE TABLE TB_ACESSO (  
    ID_ACESSO INT PRIMARY KEY,  
    ID_VISITANTE INT,  
    DATA_HORA_ACESSO TIMESTAMP,  
    TIPO_ACESSO VARCHAR(20),
```

```

        STATUS_ACESSO VARCHAR(30),
        PRIMEIRO_ACESSO TINYINT(1),
        FOREIGN      KEY      (ID_VISITANTE)      REFERENCES
TB_CADUNICO(ID_VISITANTE)
    );

```

```

-- Tabela: TB_LOGTEMP
CREATE TABLE TB_LOGTEMP (
    ID_LOG INT PRIMARY KEY,
    DATA_HORA_TENTATIVA TIMESTAMP,
    IMAGEM_TENTATIVA BLOB,
    RESULTADO_TENTATIVA VARCHAR(50),
    OBSERVACAO TEXT,
    ID_VISITANTE_ASSOCIADO INT,
    FOREIGN      KEY      (ID_VISITANTE_ASSOCIADO)      REFERENCES
TB_CADUNICO(ID_VISITANTE)
    );

```

4.4 APLICAÇÃO WEB

Tem como objetivo ser um painel de administração: interface para visualizar rostos cadastrados, adicionar/remover usuários, ver estatísticas.

Interface Gráfica (Replit ou Aplicativo Web)

Usado para visualização dos dados, gerenciamento de cadastros e acessos.

Ações:

Envia comandos de INSERT, DELETE, SELECT ao banco.

Mostra os dados para o administrador.

```

<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sistema de Acesso</title>
    <link      rel="stylesheet"      href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css" />
    <style>

```

```
* {  
  box-sizing: border-box;  
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;  
}
```

```
body {  
  margin: 0;  
  display: flex;  
  height: 100vh;  
  overflow: hidden;  
}
```

```
.sidebar {  
  width: 250px;  
  background-color: #0d6efd;  
  color: white;  
  display: flex;  
  flex-direction: column;  
  padding-top: 20px;  
}
```

```
.sidebar h2 {  
  text-align: center;  
  font-size: 22px;  
  margin-bottom: 30px;  
}
```

```
.sidebar button {  
  background: none;  
  border: none;  
  color: white;  
  padding: 15px 20px;  
  text-align: left;  
  font-size: 16px;  
  cursor: pointer;  
  display: flex;
```

```
    align-items: center;
    gap: 10px;
    transition: background 0.2s;
}
```

```
.sidebar button:hover {
    background-color: rgba(255, 255, 255, 0.1);
}
```

```
main {
    flex: 1;
    background-color: #f8f9fa;
    padding: 30px;
    overflow-y: auto;
}
```

```
h1 {
    margin-top: 0;
}
```

```
.tela {
    display: none;
}
```

```
.ativa {
    display: block;
}
```

```
.busca {
    display: flex;
    gap: 10px;
    margin-bottom: 20px;
    flex-wrap: wrap;
}
```

```
.busca input {
```

```
flex: 1;
padding: 10px;
border-radius: 6px;
border: 1px solid #ccc;
}
```

```
.busca button {
padding: 10px 20px;
border-radius: 6px;
border: none;
background-color: #0d6efd;
color: white;
cursor: pointer;
}
```

```
table {
width: 100%;
border-collapse: collapse;
background-color: white;
box-shadow: 0 2px 6px rgba(0,0,0,0.05);
}
```

```
th, td {
padding: 12px 15px;
border: 1px solid #ddd;
text-align: left;
}
```

```
.gaveta {
position: fixed;
top: 0;
right: -420px;
width: 400px;
height: 100%;
background: #fff;
box-shadow: -2px 0 10px rgba(0,0,0,0.2);
}
```

```
padding: 30px;
transition: right 0.3s ease;
overflow-y: auto;
z-index: 999;
}
```

```
.gaveta.aberta {
  right: 0;
}
```

```
.gaveta input {
  width: 100%;
  padding: 10px;
  margin-bottom: 15px;
  border-radius: 6px;
  border: 1px solid #ccc;
}
```

```
.gaveta button {
  padding: 10px 20px;
  border-radius: 6px;
  border: none;
  background-color: #0d6efd;
  color: white;
  cursor: pointer;
  margin-right: 10px;
}
```

```
.gaveta p {
  font-size: 14px;
  color: #666;
}
```

```
img {
  border-radius: 8px;
  margin-top: 10px;
```



```

        max-width: 100%;
        height: auto;
    }
</style>
</head>
<body>

<div class="sidebar">
    <h2><i class="fas fa-door-open"></i> Acesso+</h2>
    <button    onclick="mostrarTela('home')"><i    class="fas    fa-home"></i>
Home</button>
    <button    onclick="mostrarTela('cadastro')"><i    class="fas    fa-users"></i>
Cadastro</button>
    <button    onclick="mostrarTela('relatorios')"><i    class="fas    fa-chart-line"></i>
Relatórios</button>
</div>

<main>
    <section id="home" class="tela ativa">
        <h1>Bem-vindo ao Sistema de Acesso</h1>
        <p>Estamos desenvolvendo uma solução de controle de acesso inclusiva com
reconhecimento facial para cadeirantes.</p>
    </section>

    <section id="cadastro" class="tela">
        <h1>Cadastro de Usuários</h1>
        <div class="busca">
            <input type="text" id="buscaInput" placeholder="Buscar por nome ou e-
mail">
            <button onclick="buscarUsuario()">Buscar</button>
            <button onclick="abrirGaveta()">Criar</button>
        </div>
        <table>
            <thead>
                <tr>
                    <th>Nome Completo</th><th>E-mail</th><th>ID</th><th>Status</th>

```

```
    </tr>
  </thead>
  <tbody id="tabelaUsuarios"></tbody>
</table>
</section>
```

```
<section id="relatorios" class="tela">
  <h1>Relatórios de Acesso</h1>
  <p>Visualização de acessos e estatísticas (ilustração):</p>
  
</section>
</main>
```

```
<div class="gaveta" id="gavetaCadastro">
  <h2>Novo Usuário</h2>
  <input type="text" id="nome" placeholder="Nome Completo *" required>
  <input type="email" id="email" placeholder="E-mail *" required>
  <input type="text" id="id" placeholder="Código ID *" required>
  <input type="file" id="foto" required>
  <p>A inclusão de foto é obrigatória para reconhecimento facial.</p>
  <button onclick="salvarUsuario()">Salvar</button>
  <button onclick="fecharGaveta()">Fechar</button>
</div>
```

```
<script>
  const usuarios = [
    { nome: "Ana Beatriz Ferreira", email: "ana.ferreira@empresaX.com", id:
"102384", status: "Ativo" },
    { nome: "Lucas Henrique Almeida", email: "lucas.almeida@empresaX.com",
id: "104572", status: "Ativo" },
    { nome: "Thiago Ramos Pereira", email: "thiago.pereira@empresaX.com", id:
"100267", status: "Inativo" }
  ];
```

```
function mostrarTela(tela) {
  document.querySelectorAll('.tela').forEach(el => el.classList.remove('ativa'));
  document.getElementById(tela).classList.add('ativa');
}
```

```
function abrirGaveta() {
  document.getElementById("gavetaCadastro").classList.add("aberta");
}
```

```
function fecharGaveta() {
  document.getElementById("gavetaCadastro").classList.remove("aberta");
}
```

```
function renderizarUsuarios(lista = usuarios) {
  const tabela = document.getElementById("tabelaUsuarios");
  tabela.innerHTML = "";
  lista.forEach(u => {
    tabela.innerHTML +=
    <tr><td>${u.nome}</td><td>${u.email}</td><td>${u.id}</td><td>${u.status}</td></tr>;
  });
}
```

```
function salvarUsuario() {
  const nome = document.getElementById("nome").value;
  const email = document.getElementById("email").value;
  const id = document.getElementById("id").value;
  if (!nome || !email || !id) {
    alert("Preencha todos os campos obrigatórios.");
    return;
  }
  usuarios.push({ nome, email, id, status: "Ativo" });
  renderizarUsuarios();
  fecharGaveta();
  document.getElementById("nome").value = "";
  document.getElementById("email").value = "";
  document.getElementById("id").value = "";
}
```

```
document.getElementById("foto").value = "";  
}
```

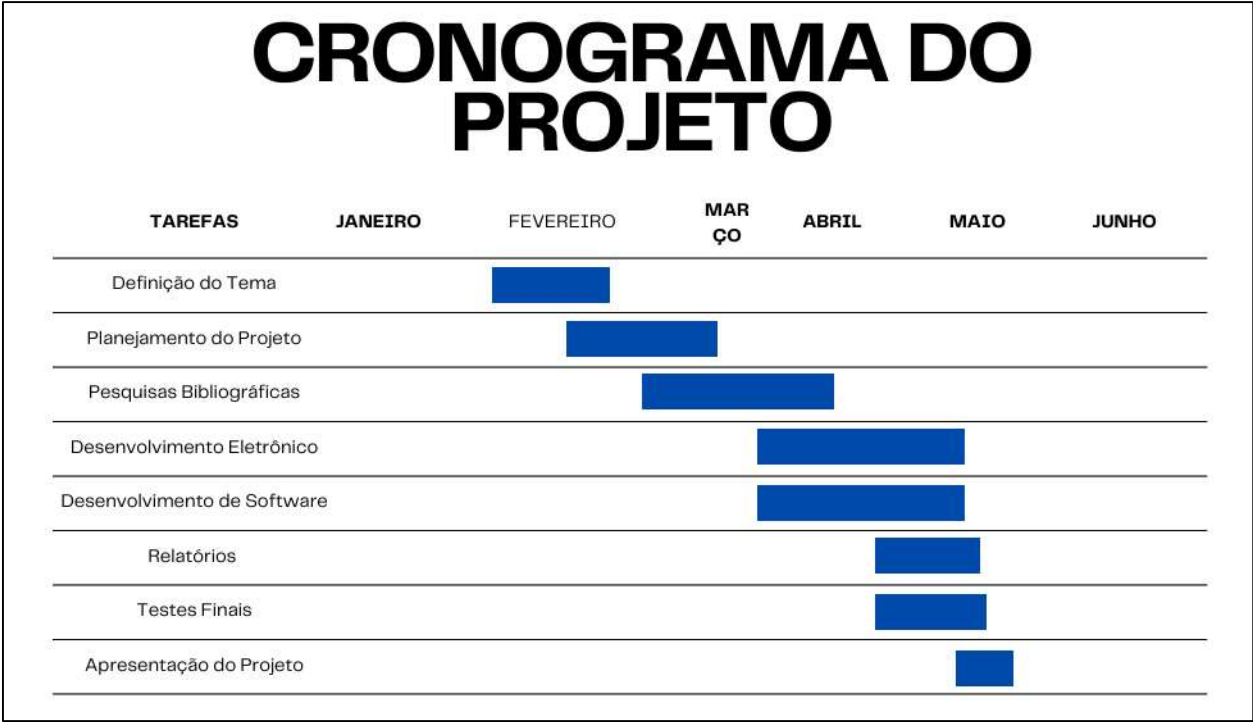
```
function buscarUsuario() {  
  const termo = document.getElementById("buscaInput").value.toLowerCase();  
  const resultado = usuarios.filter(u => u.nome.toLowerCase().includes(termo) ||  
u.email.toLowerCase().includes(termo));  
  renderizarUsuarios(resultado);  
}
```

```
renderizarUsuarios();  
</script>
```

```
</body>  
</html>
```

5. CRONOGRAMA

O cronograma está dividido em Definição do Tema, Planejamento do Projeto, Pesquisa Bibliográfica, Desenvolvimentos Eletrônicos e de Software, Relatórios, Testes e Apresentação.



6. CONCLUSÃO

O desenvolvimento da catraca inteligente para controle de acesso de pessoas com deficiência representa uma aplicação concreta de tecnologia a serviço da inclusão. Ao integrar hardware (ESP32, motores, sensores), software (Python, OpenCV, Azure) e infraestrutura digital (banco de dados em nuvem, interface web), o projeto se destaca pela inovação técnica e pelo compromisso com a acessibilidade universal. A escolha do reconhecimento facial como método de autenticação traz vantagens em segurança e comodidade, além de eliminar barreiras físicas muitas vezes impostas por sistemas tradicionais.

A estrutura modular da catraca, planejada em MDF com encaixes precisos via corte a laser, demonstra um cuidado com a viabilidade prática e com o baixo custo de produção. A integração entre ESP32 e o banco de dados garante um fluxo de decisão eficiente, permitindo tanto a operação local quanto a escalabilidade para ambientes corporativos e educacionais. A realização de testes e a validação com usuários reais reforçam a preocupação com usabilidade e eficiência da solução. Em suma, este projeto vai além de uma simples catraca automatizada: ele propõe uma experiência de acesso segura, inclusiva e tecnológica para todos.

7. REFERÊNCIAS

PINAGEM DA RESPOSTA DO SENSOR PARA A PLACA OPS010

Centro Universitário Senac. *Pinagem da resposta do sensor para a placa OPS010*. [2025]. Disponível em:

[https://senacsp.blackboard.com/ultra/courses/_263994_1/cl/outline].

Especificações Técnicas OPS010 Oppler

Centro Universitário Senac. *Especificações técnicas OPS010 Oppler.*, [2025].

Disponível em: [https://senacsp.blackboard.com/ultra/courses/_263994_1/cl/outline].

MANUAL DE USO PRÁTICO

Centro Universitário Senac. *Manual de uso prático*. [2025]. Disponível em:

[https://senacsp.blackboard.com/ultra/courses/_263994_1/cl/outline].